



《系统软件原理与设计III-数据库系统》

第7章 并发控制

计算机科学与工程学院（网络空间安全学院）

任课教师：胡旺

2025年12月16日星期二



本章内容



1

事务并发

2

并发事务引起的问题

3

可串行化

4

基于锁的并发控制协议

5

*活锁与死锁

6

*多粒度封锁



■ I/O与CPU等可以并行交叉运行

■ 并发执行的优点

- ◆ 改善系统的资源利用率
- ◆ 减少短事务的等待时间

■ 调度 (schedule)

- ◆ 一个或多个事务的操作按时间排序的一个序列。

T ₁	T ₂
READ(A) WRITE(A)	
	READ(C) WRITE(C)
RAED(B) WRITE(B)	

一个事务的两个操作在调度中出现的顺序必须与其在事务内定义的先后顺序一致。



本章内容



1

事务并发



2

并发事务引起的问题

3

可串行化

4

基于锁的并发控制协议

5

*活锁与死锁

6

*多粒度封锁



■ 读脏数据 (dirty read)

◆ 脏数据 (dirty data) 是对未提交事务所写数据的统称。

T1	T2
READ(A) $A := A + A * 0.1$ WRITE(A)	
	READ(A) COMMIT
ROLLBACK	

- ✓ 若脏读就造成了数据库的不一致状态，应严格禁止。
- ✓ 若脏读带来的影响足够小，偶尔可读一次脏数据，它可以提高并发性，减少事务的等待时间



■ 不可重复读 (unrepeatable read)

T1	T2
READ(A)	
READ(B)	
READ(C)	
READ(D)	READ(A)
READ(E)	$A := A + A * 0.1$
.....	WRITE(A)
.....	COMMIT
READ(A)	
.....	
.....	

➤ 事务T1的两次读取数据之间，其它事务修改了它要读取的数据，以致两次读到的值不同

➤ 在事务串行执行时，不会出现此现象



■ 丢失更新 (lost update)

T1	T2
READ(A)	READ(A)
$A := A + A * 0.1$	$A := A + 10$
WRITE(E)	WRITE(A)
	COMMIT
COMMIT	

➤ 由两个事务对同一数据并发地写入引起



本章内容



1

事务并发

2

并发事务引起的问题



3

可串行化

4

基于锁的并发控制协议

5

*活锁与死锁

6

*多粒度封锁



■ 回顾：事务ACID特性中的隔离性？

- **事务在运行中不受其它事务干扰的方法：**
 - **串行：**
 - ◆ 每个事务依次顺序执行
 - **并行但控制：**
 - ◆ 事务之间并发执行，DBMS调整事务的调度，使其运行结果与一次只执行一个事务的结果相同



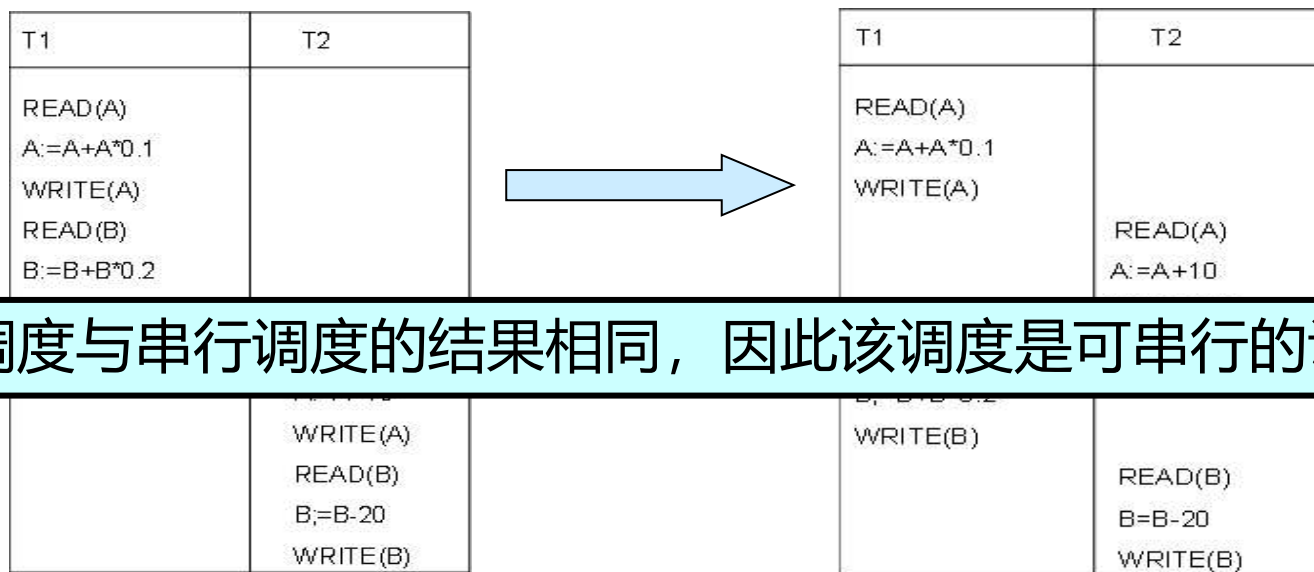
- | T1 | T2 |
|--|--|
| READ(A)
A:=A+A*0.1
WRITE(A)
READ(B)
B:=B+B*0.2
WRITE(B) | READ(A)
A:=A+10
WRITE(A)
READ(B)
B:=B-20
WRITE(B) |

DATABASE@UESTC



■ 可串行化调度

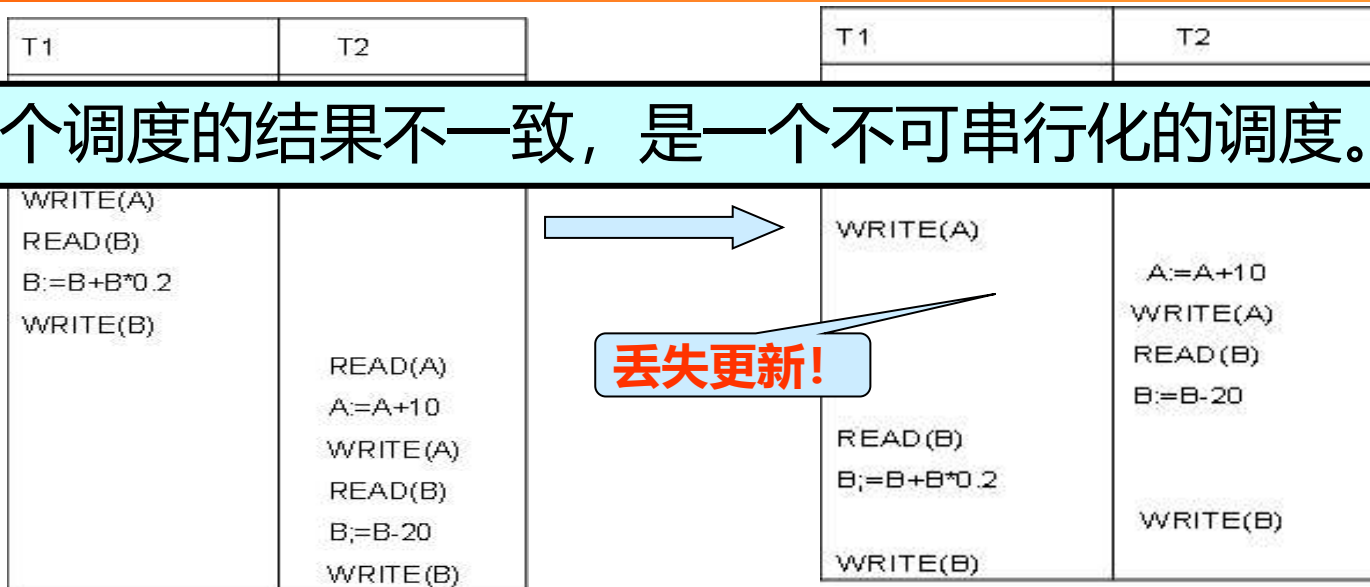
- ◆调度是可串行化的：多个事务交叉调度的结果与某一个串行调度的结果相同
- ◆DBMS认为事务串行调度的结果保持了数据库的一致性，都是正确的
- ◆一个调度如果是可串行化的，系统认为其调度是一个正确的调度，保持了数据库的一致性



➤并行调度与串行调度的结果相同，因此该调度是可串行的调度



➤两个调度的结果不一致，是一个不可串行化的调度。



■ DBMS需要事务调度管理

- ◆如果将事务的并发执行完全交给操作系统，则任何一种调度方式都有可能出现。
- ◆有的调度能保持数据库的一致，有的调度却会产生错误的结果。
- ◆DBMS必须对事务的运行加以控制，确保交叉调度完毕后的结果与某一串行调度的结果相同，数据库不会出现不一致的状态。



■ 简记符号

- ◆ WRITE简写为W,
- ◆ READ简写为R,
- ◆ WT (X) : 事务T写数据库元素X,
- ◆ RT (X) : 事务T读数据库元素X,
- ◆ S表示一个调度。

■ 调度(事务序列)表示:

- ◆ $S = R1(A) \ R2(A) \ W1(A) \ W2(A) \ R2(B) \ R1(B) \ W2(B) \ W1(B)$



调度中两个事务发生冲突，必须：

- 对同一数据对象进行操作
- 两个操作指令中有一个是写操作W

■ 指令冲突性

◆ 读相同数据：不冲突

- 若事务 T_i 和 T_j 都是读取数据A，则 $R_i(A)$ ， $R_j(A)$ 指令不发生冲突。

◆ 读写相同数据：冲突

- 若事务 T_i 和 T_j 一个是读数据，一个是写数据，则事务的执行顺序是重要的。 $R_i(A)$ 和 $W_j(A)$ 指令是冲突的。

◆ 写相同数据：冲突

- 若事务 T_i 和 T_j 都是写数据A，则 $W_i(A)$ 和 $W_j(A)$ 指令也是冲突的。

◆ 读写不同数据：不冲突

■ 示例

◆ $S = R_1(A) \ R_2(A) \ W_1(A) \ W_2(A) \ R_2(B) \ R_1(B) \ W_2(B) \ W_1(B)$

◆ T2事务的READ(A)与T1事务的WRITE(A)是冲突指令

◆ T1事务的READ(A)与T2事务的READ(A)指令是不冲突。



■ 冲突等价：

- ◆ 若调度S中属于不同事务的两条操作指令是不冲突的，则可以交换两条指令的执行顺序，得到一个新的调度S'。称调度S与调度S' 冲突等价的（conflict equivalent）。

■ 冲突可串行化：

- ◆ 若一个调度冲突等价于一个串行调度，则该调度是冲突可串行化的。

■ 示例

- ◆ 调度S= R1 (A) W1 (A) R2 (A) W2 (A) R1 (B) W1 (B) R2 (B) W2 (B)
- ◆ R1 (B) 与W2 (A) 指令不冲突，可以交换执行顺序；
- ◆ R1 (B) 与R2 (A) 指令不冲突，可以交换执行顺序；
- ◆ W1 (B) 与W2 (A) 指令不冲突，可以交换执行顺序；
- ◆ W1 (B) 与R2 (A) 指令不冲突，可以交换执行顺序。
- ◆ 调度S' = R1 (A) W1 (A) R1 (B) W1 (B) R2 (A) W2 (A) R2 (B) W2 (B)

➤ 调度S'是一个串行调度。

➤ 调度S等价于串行调度S'，是冲突可串行化的。



■ 冲突可串行是可串行性的充分条件

T1	T2	T3
READ(A) A:=A+A*0.1 WRITE(A)	READ(A) A:=A+10 WRITE(A) READ(B) B:=B-20 WRITE(B)	
READ(B) B=B+B*0.2 WRITE(B)		WRITE(B)

➤ 调度运行结果与串行调度T1→T2→T3的运行结果是一致的，但调度不是冲突可串行的



◆对同一事务集，如果两个调度S1和S2在任何时候都保证每个事务读取相同的值，写入数据库的最终状态也是一样的，则称调度S1和S2视图等价。

T1	T2
READ(A) A:=A+A*0.1 WRITE(A) READ(B) B:=B+B*0.2 WRITE(B)	READ(A) A:=A+10 WRITE(A) READ(B) B:=B-20 WRITE(B)

- 调度S1和调度S2不是视图等价的
 - 调度S1中T2事务读取的A值是事务T1修改后的值,
 - 调度S2中T2事务读取的A值是事务T1修改前的值。



■ 视图可串行化

◆ 如果某个调度视图等价于一个串行调度，则称这个调度是视图可串行化的

■ 如果调度是冲突可串行化的，则该调度一定是视图可串行化的。但反过来未必成立。

■ 举例

◆ 设调度 $S1 = R1(A) W3(A) R2(B) W1(B)$

◆ 经过非冲突调整， $S2 = R2(B) R1(A) W1(B) W3(A)$

◆ 调度 $S1$ 和调度 $S2$ 是冲突等价的。

◆ 又因为调度 $S2$ 为一串行调度，因此调度 $S1$ 是冲突可串行化的。

◆ 对于调度 $S1$ 和 $S2$ ，事务 $T1$ 读取的 A 、事务 $T2$ 读取的 B 都是数据库的初始值；数据库最终的 A 、 B 值都是由事务 $T3$ 和 $T1$ 写入的。

◆ 因此，调度 $S1$ 和 $S2$ 是视图可串行化的。



■ 判定一个调度是否是冲突可串行化的，可以使用前驱图（precedence graph）

- ◆ 若前驱图中存在环，则表示调度S是不可串行化的。
- ◆ 反之，若前驱图中不存在环，表示调度S是冲突可串行化的，可用拓扑排序得到调度S 的一个等价的串行调度。

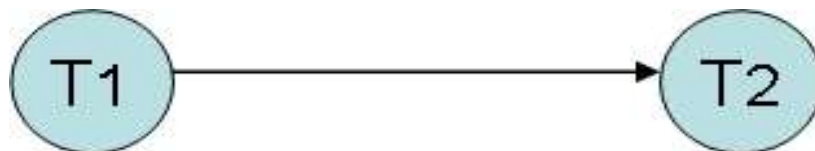
- 前驱图是一个有向图 $G = (V, E)$
- 顶点代表调度S 中的事务
- 由 $T_i \rightarrow T_j$ 的边表示在调度S中 T_i 和 T_j 之间存在一对冲突指令，并且 T_i 中的指令先于 T_j 中的指令执行。



■ 示例1

◆ $S = R1(A) \text{ } W1(A) \text{ } R2(A) \text{ } W2(A) \text{ } R1(B) \text{ } W1(B) \text{ } R2(B) \text{ } W2(B)$

◆ 冲突指令 $W1(A)$ 在 $R2(A)$ 前, $W1(B)$ 在 $R2(B)$ 前, 因此存在从 $T1$ 到 $T2$ 的有向边。



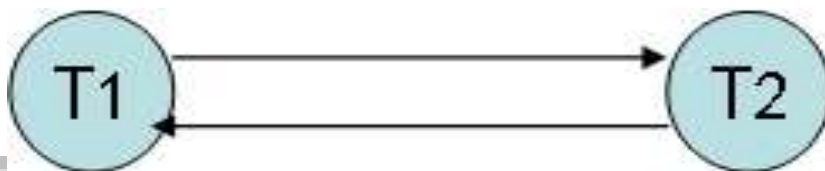
如何设计算法并实现?

■ 示例2

◆ $S = R1(A) \text{ } \underline{R2(A)} \text{ } \underline{W1(A)} \text{ } \underline{W2(A)} \text{ } R2(B) \text{ } R1(B) \text{ } \underline{W2(B)} \text{ } \underline{W1(B)}$

◆ $W1(A)$ 在 $W2(A)$ 之前, $R1(B)$ 在 $W2(B)$ 之前, 因此存在 $T1$ 到 $T2$ 的有向边;

◆ $R2(A)$ 在 $W1(A)$ 之前, $W2(B)$ 在 $W1(B)$ 之前, 因此存在 $T2$ 到 $T1$ 的有向边。





■ 数据库系统要求所有的调度都是可恢复的

T1	T2
READ(A)	
A:=A*0.1	
WRITE(A)	
	READ(A)
	COMMIT
ROLLBACK	

可恢复条件:

- 调度S中, 事务 T_i 如果读取了事务 T_j 修改过的数据, 则事务 T_i 必须等事务 T_j 提交后才能提交。



- 事务在并行执行过程中发生故障，还可能引起多个事务的级联回滚。

【例：级联读脏】

- 假定 T_2 事务读取 A 的值并修改；
- 还有 T_3 事务读取 T_2 修改后的值，并做了修改；依次类推。。。
- 若事务 T_1 发生故障时，后续的事务 T_2 、 T_3 、 $T_4 \dots$ 都已提交，则事务 T_1 的回滚导致级联回滚，产生大量的撤销工作。

无级联回滚的调度应满足：

- 调度 S 中的每对事务 T_i 和 T_j ，事务 T_i 如果读取了事务 T_j 修改过的数据，则事务 T_j 必须在 T_i 读取前提交
- 即调度禁止读取脏数据。



本章内容



1

事务并发

2

并发事务引起的问题

3

可串行化



4

基于锁的并发控制协议

5

*活锁与死锁

6

*多粒度封锁



■ 封锁

- ◆指事务在对数据库进行读、写操作之前，必须先得到对操作对象的控制权力。
- ◆先要对将执行读、写操作的数据库对象申请锁，在获得该数据库对象的控制权力后，才能进行相应地读、写操作。
- ◆封锁是实现数据库并发控制的重要手段。

如何设计算法并实现？

■ 锁管理器 (lock manager)

- ◆事务执行过程中锁的申请和释放由DBMS中的锁管理器负责
- ◆锁管理器维护一张哈希表——锁表
- ◆对每个数据库对象，如果其上有锁，那么锁表指明持有该锁的事务。
- ◆锁表包含的信息包括：每个数据库对象上已有的锁的个数、锁的类型以及一个指向申请锁队列的指针。



锁的类型

- ◆共享锁（S锁）：如果事务 T_i 申请到数据项 Q 的共享锁，则 T_i 可以读数据项 Q ，但不能写 Q 。
- ◆排它锁（X锁）：如果事务 T_i 申请到数据项 Q 的排它锁，则 T_i 可以读数据项 Q ，也可以写 Q 。

锁的相容性

		已分配的锁	
		T2	
申请锁	T1	S	X
S	Y	N	
X	N	N	

Y 相容
N 不相容



■ 当事务需要操作数据项时，它向锁管理器发出锁的申请：

- ◆ 若申请的是一个共享锁，且申请队列为空，当前数据项上也没有排它锁，则锁管理器授予锁，并修改数据项的锁表。
- ◆ 若申请的是一个排它锁，当前也没有其它的事务拥有该数据项上的锁，则锁管理器授予锁，并修改数据项的锁表。
- ◆ 否则，申请的锁不能马上授予，锁申请加入申请队列，申请锁的事务挂起。



锁协议

申请锁



T1	T2	T1	T2	T1	T2
<code>XLOCK(A)</code> <code>READ(A)</code> <code>A=A+A*0.1</code> <code>WRITE(A)</code> <code>UNLOCK(A)</code>		<code>XLOCK(A)</code> <code>READ(A)</code> <code>A=A+A*0.1</code> <code>WRITE(A)</code>		<code>XLOCK(A)</code> <code>READ(A)</code> <code>A=A+A*0.1</code> <code>WRITE(A)</code>	
	<code>XLOCK(A)</code> <code>READ(A)</code> <code>A=A+10</code> <code>WRITE(A)</code> <code>UNLOCK(A)</code>	<code>XLOCK(B)</code> <code>READ(B)</code> <code>B:=B+B*0.2</code> <code>WRITE(B)</code> <code>UNLOCK(A)</code> <code>UNLOCK(B)</code>	<code>XLOCK(A)</code> 等待 ...		<code>XLOCK(B)</code> <code>READ(B)</code> <code>B=B-20</code> <code>WRITE(B)</code> <code>XLOCK(A)</code> 等待
<code>XLOCK(B)</code> <code>READ(B)</code> <code>B=B+B*0.2</code> <code>WRITE(B)</code> <code>UNLOCK(B)</code>			 <code>READ(A)</code> <code>A=A+10</code> <code>WRITE(A)</code> <code>XLOCK(B)</code> <code>READ(B)</code> <code>B:=B-20</code> <code>WRITE(B)</code> <code>UNLOCK(A)</code> <code>UNLOCK(B)</code>	<code>XLOCK(B)</code> 等待	
	<code>XLOCK(B)</code> <code>READ(B)</code> <code>B:=B-20</code> <code>WRITE(B)</code> <code>UNLOCK(B)</code>				

(a)

更新后立即释放锁，
可能脏读。

(b)

事务的最后释放锁，
避免脏读和确保可串
行性。但降低并发度

(c)

相互等待出现死锁



- 两段锁协议（two-phase locking protocol, 2PL）是指所有事务分两个阶段提出加锁和解锁申请：
 - ◆ 增长阶段（growing phase）：在对任何数据进行读、写操作之前，首先申请并获得该数据的封锁；
 - ◆ 收缩阶段（shrinking phase）：在释放一个封锁后，事务不再申请和获得其它的任何封锁。
- 两段锁协议是保证冲突可串行化的充分条件，但该协议不保证不发生死锁。

集中申请
有序申请



■ 两段锁协议的级联回滚现象

T1	T2	T3
XLOCK(A) READ(A) $A := A + A * 0.1$ WRITE(A) XLOCK(B) UNLOCK(A) READ(B) $B := B + B * 0.2$ WRITE(B) UNLOCK(B)	XLOCK(A) READ(A) $A := A + 10$ WRITE(A) UNLOCK(A)	SLOCK(A) READ(A)

- 每个事务都遵从两段锁协议；
- 若T1事务在WRITE (B) 时刻发生故障，将导致事务T2、T3级联回滚。



■ 严格两阶段锁

- ◆ 除要求满足两段锁协议规定外，还要求事务的**排它锁**必须在事务提交之后释放。
- ◆ 解决级联回滚问题
- ◆ 避免了脏读和丢失修改的问题。

■ 强两阶段锁

- ◆ 除要求满足两段锁协议规定外，还要求事务的**所有锁**都必须在事务提交之后释放。
- ◆ 进一步解决数据项不能重复读的问题



■ 两阶段锁总结

- ◆ 从两段锁协议到严格两段锁协议，再到强两段锁协议，事务持锁的时间不断增长。这不但保证事务的并发调度是冲突可串行化的，还不断增强了一致性保证。
- ◆ 但带来的另一方面的问题是并发度的降低，以及死锁出现可能性的增加。
- ◆ 目前，大多数的DBMS都采用严格两段锁协议或强两段锁协议。





■ 更新锁相容矩阵

		申请锁		
已分配的锁	T1 \ T2	S	X	U
	S	Y	N	Y
	X	N	N	N
	U	N	N	N

Y 相容
N 不相容

已持有共享锁，可以再申请共享锁或更新锁



本章内容



1

事务并发

2

并发事务引起的问题

3

可串行化

4

基于锁的并发控制协议



5

*活锁与死锁

6

*多粒度封锁



■ 活锁或饿死

T1	T2	T3	T4	T5
SLOCK(A) READ(A)	XLOCK(A) 等待			
	等待	SLOCK(A) READ(A)		
UNLOCK(A)	等待		SLOCK(A) READ(A)	
	等待		UNLOCK(A)	SLOCK(A) READ(A)
	等待	UNLOCK(A)		

➤解决活锁方法：
采用先来先服务的策略。



■ 死锁

T1	T2
SLOCK(A) READ(A)	XLOCK(B) READ(B)
XLOCK(B) 等待	XLOCK(A) 等待

死锁的两种处理方式：

- 一种是进行死锁的预防，不让并发执行的事务出现死锁的状况；
- 一种是允许死锁的发生，在死锁出现后采取措施解决，为此系统中需增加死锁的检测及死锁的解除算法



■ 顺序封锁法

- ◆ 将数据库对象按某种规定的顺序排列，要求事务实行封锁也必须按照这个顺序进行。
- ◆ 缺点：
 - 不好确定数据库对象的封锁顺序。
 - 维护封锁顺序是件困难的事情且成本很高

■ 一次封锁法

- ◆ 要求事务在开始执行前先申请到所需的所有封锁，如果有一个封锁没有申请到，则事务中止。
- ◆ 缺点：
 - 在事务开始前很难预先知道哪些数据项需要封锁；
 - 一次将所有需要的封锁申请到，可能有些封锁只在事务运行的后期才需要，这就大大降低了系统的并发度。



预防更有效，如腐败预防、自然灾害防御

■ 时间戳法

- ◆ 根据事务启动时的时间戳设置事务的优先级，越早开始运行的事务优先级越高。
- ◆ 为预防死锁，在事务 T_i 申请的封锁与事务 T_j 已经拥有的封锁发生冲突时，锁管理器可使用如下两种不同的机制：
- ◆ **Wait-die机制**：若 T_i 优先级较高，则 T_i 可以等待；否则中止事务 T_i 。
- ◆ **Wound-wait机制**：若 T_i 优先级较高，则中止 T_j ；否则 T_i 等待。

■ 示例：假设事务 T_1 、 T_2 、 T_3 的时间戳分别为5，10，20。

- ◆ 在Wait-die机制下：
 - 若 T_1 申请的封锁被 T_2 拥有，则 T_1 等待；
 - T_3 申请的封锁被 T_2 拥有，则 T_3 中止运行做回滚操作。
- ◆ 在Wound-wait机制下：
 - 若 T_1 申请的封锁被 T_2 拥有，则中止事务 T_2 的运行；
 - 若 T_3 申请的封锁被 T_2 拥有，则 T_3 等待。

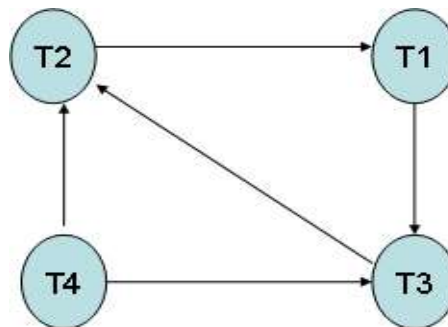
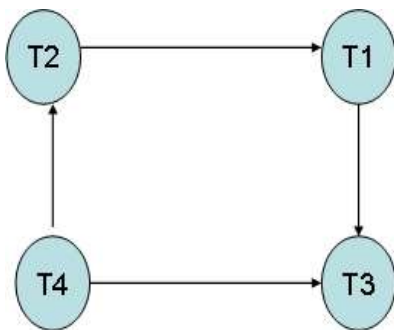


■ 超时法：

- ◆ 规定申请锁事务等待的最长时间。若超过了规定时间，则系统判定出现死锁，此时该事务本身回滚并重启。
- ◆ 实现简单
- ◆ 可能出现误判
- ◆ 等待多长时间合适难以把握

■ 等待图法：

- ◆ 当且仅当等待图中出现环路时，表示系统中存在死锁。





■ 死锁的解除

- ◆ 选择一个或多个事务撤销，释放这个或这些事务拥有的封锁。

■ 撤销事务的选择。

- ◆ 为解除死锁必须回滚处于死锁状态的部分事务。
- ◆ 撤销事务的选择原则是事务撤销所需的系统代价最小。

■ 事务撤销的程度。

- ◆ 全部回滚选中事务，然后重新开始。
- ◆ 部分回滚选中事务，需要系统维护更多的事务运行状态信息。



本章内容



1

事务并发

2

并发事务引起的问题

3

可串行化

4

基于锁的并发控制协议

5

*活锁与死锁



6

*多粒度封锁



■ 封锁对象的大小称为封锁粒度（granularity）

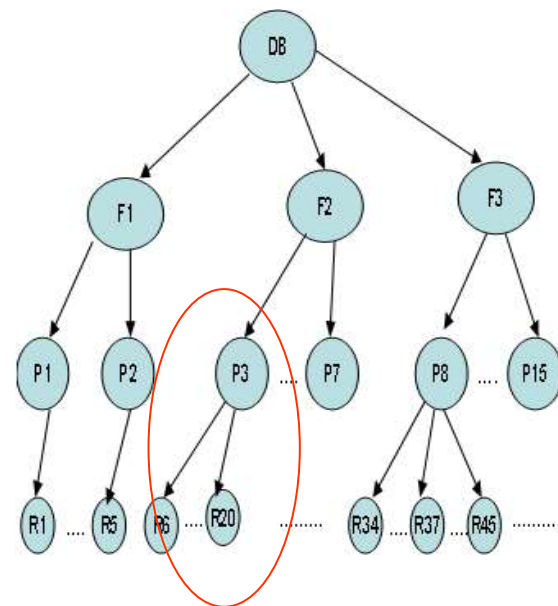
■ 封锁对象：

- ◆ 数据库的逻辑单位，如属性、元组、关系、索引、数据库等；
- ◆ 数据库的物理单位，如页、块等；

■ 封锁粒度对并发度和资源消耗的影响：

- ◆ 若封锁粒度小，则系统并发度高，资源消耗多；
- ◆ 若封锁粒度大，则系统并发度低，资源消耗小；

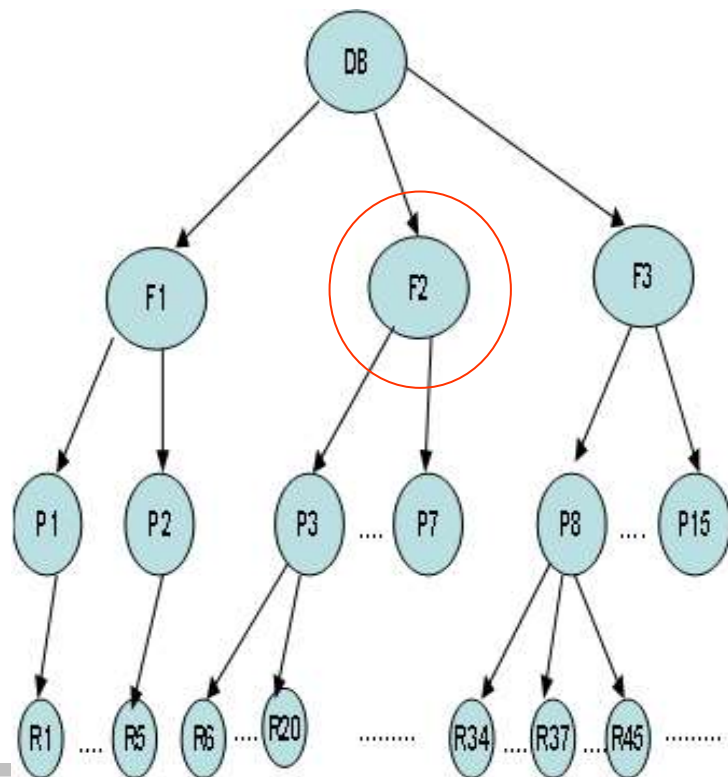
■ 不同事务可能需要不同的封锁粒度，系统允许同时为不同的事务提供不同的封锁粒度选择。





■ 意向锁 (intention lock)

- ◆ 如果对一个结点加意向锁，则意味着要对该结点的所有子孙结点显式加锁；
- ◆ 在一个结点显式加锁前，该结点的所有祖先结点都应加上意向锁。





■ 意向锁类型

- ◆ 意向共享锁（IS锁）：如果一个结点加IS锁，那么将在该结点的子孙结点进行显式封锁，加S锁
- ◆ 意向排它锁（IX锁）：如果一个结点加IX锁，那么表示将在该结点的子孙结点进行显式封锁，可以加排它锁和共享锁
- ◆ 共享意向排它锁（SIX锁）：若一个结点加SIX锁，那么将对该结点的子结点显式地加共享锁，对更低层的结点显式地加排它锁

■ 意向锁的相容性

		已分配的锁					
		T2					
		T1 \	S	X	IS	IX	SIX
申请锁	S	Y	N	Y	N	N	
	X	N	N	N	N	N	
	IS	Y	N	Y	Y	Y	
	IX	N	N	Y	Y	N	
	SIX	N	N	Y	N	N	

Y 相容
N 不相容



本章小结



- 事务并发引起的问题
- 调度、可串行化调度
- 指令冲突性、冲突等价、冲突可串行化、视图可串行化
- 锁、两阶段锁协议
- 活锁、死锁、死锁预防、死锁检测、死锁解除
- 封锁粒度